

LECTURE 18

Normalization: Purpose and Concepts

Why some relation designs quietly corrupt themselves — and how to know

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

From Redundancy to a Fixable Problem

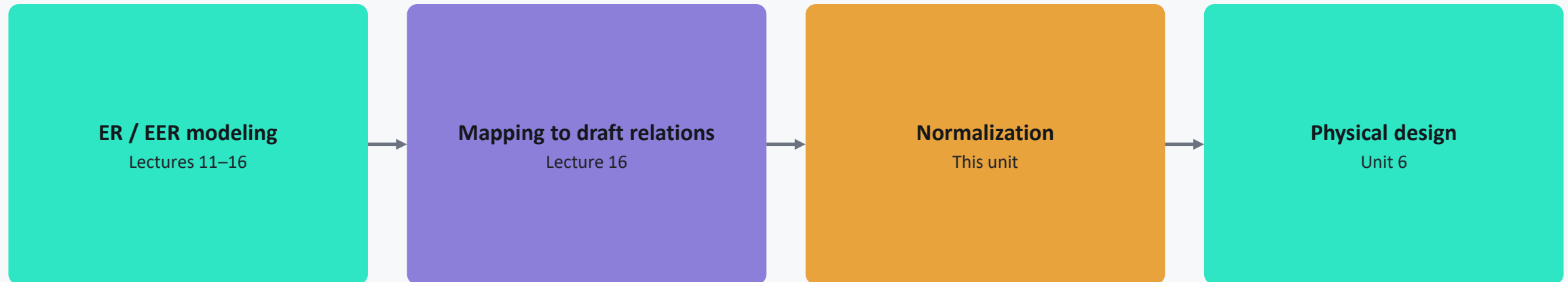
- 1 What normalization is for — not “make it neat,” but eliminate provable problems
- 2 Data redundancy: a concrete, worked example of a poorly designed relation
- 3 The three update anomalies — insertion, deletion, modification
- 4 Functional dependencies: the notation $X \rightarrow Y$ that makes “good design” checkable
- 5 Decomposition, and its two required properties: lossless-join & dependency-preservation
- 6 The overall shape of the normalization process — 1NF through BCNF/4NF

What Normalization Is For

Normalization is a formal technique for analyzing a relation based on its functional dependencies, and systematically decomposing it into smaller relations that are provably free of certain kinds of redundancy — and the anomalies that redundancy causes.

NOTE

Normalization does not tell you WHAT entities or relationships your database needs — that's ER/EER modeling's job. It tells you whether the relations you already derived are well-structured, and how to repair them if they aren't.



THE PROBLEM

Data Redundancy: A Worked Example

A rushed designer proposes one wide relation to record orders and their products:

`Order (orderNo, orderDate, custNo, custName, custAddress, productNo, productName, unitPrice, qty)`

orderNo	custNo	custName	custAddress	productNo	productName	qty
O100	C001	Ann Beech	12 Elm St, Lahore	P01	Widget	3
O100	C001	Ann Beech	12 Elm St, Lahore	P02	Gadget	1
O101	C002	Tom Kelly	5 Oak Ave, Karachi	P01	Widget	2
O102	C001	Ann Beech	12 Elm St, Lahore	P03	Sprocket	10

Ann Beech's name/address repeats in 3 rows; **Widget**'s name/price repeats in 2 rows — the same fact, stored redundantly, because this one relation describes three different kinds of things (orders, customers, products) at once.

Insertion Anomaly

Product P04 (“Cog”) hasn't been ordered yet. **There is no way to insert this fact** into the Order relation — every row requires an orderNo, and there is no order to attach it to.

A product should be able to exist in the catalog before its first sale. Symmetrically, a brand-new customer who hasn't placed an order yet cannot be recorded either — custName and custAddress only exist attached to an orderNo.

All three anomalies trace to one root cause:

this one relation forces facts about three different kinds of things — orders, customers, and products — to live and die together, when in reality they don't.

Deletion & Modification Anomalies

Deletion anomaly — Order O101 (Tom Kelly's ONLY order) is cancelled. Deleting that row silently erases the only record this database had of Tom Kelly's existence — his name and address vanish with the order.

Modification anomaly — Widget's (P01) price changes from 9.99 to 11.49. That fact is stored in TWO rows. Updating only one leaves the relation internally inconsistent:

orderNo	productNo	productName	unitPrice
O100	P01	Widget	11.49 (updated)
O101	P01	Widget	9.99 (STALE — should be 11.49)

WARNING

The same product now has two different prices depending on which row you query — easy to trigger by accident through application code that updates only “the row the user is looking at.”

Functional Dependencies

NOTE

Given relation R and attribute sets $X, Y \subseteq R$, Y is functionally dependent on X — written $X \rightarrow Y$, read “ X functionally determines Y ” — if every legal value of X is associated with exactly one value of Y . If two tuples agree on X , they must agree on Y too. X is the determinant.

Reading the FDs actually present in the **Order** relation:

```
orderNo → orderDate, custNo  
custNo → custName, custAddress  
productNo → productName, unitPrice  
{ orderNo, productNo } → qty
```

An FD is a statement about the SCHEMA, not about today's data — it must hold for every possible legal instance, not just the sample rows shown.

Functional Dependency and Normalization

Every normal form from this point on is defined purely in terms of which FDs hold and how they relate to the relation's keys:

Normal Form	Asks...
1NF	Is every attribute value atomic? (independent of FDs)
2NF	Does every non-key attribute depend on the WHOLE key, or only part (partial dependency)?
3NF	Does every non-key attribute depend DIRECTLY on the key, or only through another non-key attribute (transitive)?
BCNF	Is EVERY determinant a superkey, with no exceptions?

Without functional dependencies, “well-designed” stays a matter of taste. With them, it becomes something you can prove.

Decomposition of Relations

Decomposition replaces one relation with two or more whose attributes, together, cover the same information — removing the partial/transitive dependencies that cause anomalies. A preview of where Order is headed (full walkthrough in Lecture 19):

Customer (custNo, custName, custAddress)

custNo	custName	custAddress
C001	Ann Beech	12 Elm St, Lahore
C002	Tom Kelly	5 Oak Ave, Karachi

Product (productNo, productName, unitPrice)

productNo	productName	unitPrice
P01	Widget	9.99
P02	Gadget	24.99
P03	Sprocket	4.50

Order (orderNo, orderDate, custNo)

orderNo	orderDate	custNo
O100	2026-08-01	C001
O101	2026-08-02	C002
O102	2026-08-03	C001

OrderLine (orderNo, productNo, qty)

orderNo	productNo	qty
O100	P01	3
O100	P02	1
O101	P01	2
O102	P03	10

Lossless-Join Property

A decomposition of R into $R_1, R_2, \dots, R_n$ has the **lossless-join property** if the natural join $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$ reconstructs exactly R — no rows lost, and no extra spurious rows gained.

```
Customer ⋈ Order ⋈ OrderLine ⋈ Product  reproduces the original Order table exactly
```

WARNING

The classic way to lose losslessness: split on the wrong attribute. If Order were split into (orderNo, custNo, productNo) and (productNo, productName, unitPrice, qty) — dividing attributes without regard to which FDs justify the split — rejoining on productNo alone would produce every combination of order and quantity for a product, most of which never happened. A correct decomposition is always guided by functional dependencies, never an arbitrary attribute split.

Dependency-Preservation Property

A decomposition has the **dependency-preservation property** if every FD from the original relation's FD set can still be checked directly on the decomposed relations — without joining them back together first.

In the Order decomposition: $\text{custNo} \rightarrow \text{custName}$ is checkable directly on Customer alone (declare custNo its PK, the DBMS enforces it automatically), and $\{\text{orderNo}, \text{productNo}\} \rightarrow \text{qty}$ is checkable directly on OrderLine. Every original FD survives onto exactly one decomposed relation — lossless AND dependency-preserving.

TIP

This matters practically: a dependency that can only be verified after a join is one the DBMS cannot enforce cheaply with a simple key constraint on one table. Lecture 20 shows this property isn't always achievable simultaneously with the strictest normal form, BCNF — a genuine trade-off.

The Normalization Process, Staged

Unnormalized (UNF)

May contain repeating groups / non-atomic values



1NF

Atomic values, no repeating groups (Lecture 19)



2NF

No partial dependency on a composite key (Lecture 19)



3NF

No transitive dependency (Lecture 19)



BCNF

Every determinant is a superkey (Lecture 20–21)



4NF

No non-trivial multi-valued dependency (Lecture 22)

KEY TAKEAWAYS

Key Takeaways

- 1 Normalization is a formal, FD-driven technique for validating and refining relations produced during ER/EER design — a check, not a replacement
- 2 Redundancy is the root cause of the three update anomalies: insertion, deletion, modification
- 3 A functional dependency $X \rightarrow Y$ formalizes “X determines Y” precisely enough to check mechanically — every later normal form is defined in terms of FDs and keys
- 4 Decomposition must satisfy the lossless-join property and, ideally, dependency-preservation
- 5 The process is fixed and increasingly strict: $UNF \rightarrow 1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF \rightarrow 4NF$, each removing one specific category of dependency problem